

PRAKTIKUM
SISTEM BASIS DATA

Nama	Jonathan Frederick Kosasih	No. Modul	5
NPM	2306225981	Tipe	TP

PART 1: Teori

1. Application Programming Interface atau (API) sebuah mekanisme yang membuat dua atau lebih sistem software, aplikasi, dataset, atau node dapat saling berkomunikasi dan bertukar informasi. API mengintegrasikan banyak aplikasi untuk berbagi informasi dengan cepat dan mudah. API membuat client dapat membuat request kepada server yang kemudian akan mengembalikan sebuah respons.

Beberapa istilah umum dalam API:

- **Endpoint:** URL tujuan dari sebuah API request yang akan memberikan akses untuk sebuah sumber daya tertentu. URL tersebut dapat terbagi menjadi beberapa bagian tergantung konfigurasi client dengan server. URL dari endpoint biasanya berisi domain dari server tujuan sampai dengan aset tertentu yang ingin diakses.
- **Method:** instruksi yang mendefinisikan operasi yang harus dilakukan server terhadap sebuah sumber daya tertentu yang sudah dispesifikasi oleh client. REST (protokol API yang paling dominan) menggunakan HTTP method standar, seperti GET, POST, PUT, PATCH, dan DELETE untuk mendapatkan, membuat, memperbarui, atau menghapus data.
- **Body:** bagian dari API request yang menyimpan objek data dan atribut yang akan dikirimkan melalui request tersebut untuk membuat, memperbarui, atau menghapus sebuah sumber daya. Body biasanya diformat ke dalam bentuk JSON dan XML.
- **Params & Query:** argumen yang berisi data tambahan untuk memfilter sebuah API request. Parameters membuat pemanggilan API menjadi lebih fleksibel dan dinamis. Parameters juga digunakan untuk menspesifikkan atribut mana yang ingin diperoleh, untuk otentikasi dan pengelolaan user, memberikan data baru dalam request kita, atau untuk tujuan transfer data lainnya. Query pada parameters ditambahkan pada akhir endpoint sebagai informasi tambahan yang diberikan.

- **Headers:** pasangan key-value dari API request yang memberikan informasi atau konteks tambahan untuk data yang dikirimkan melalui request tersebut. Headers digunakan untuk membantu otentikasi, menyimpan cookies, dan memberikan informasi tambahan.
- **Status:** merupakan sebuah kode 3 digit yang mengindikasikan apa yang telah dilakukan oleh server terhadap request kita. Digit pertama kode mengindikasikan kategori statusnya, sedangkan 2 digit lainnya mengindikasikan status spesifiknya. Kode status berkepala 2 mengindikasikan request berhasil, kode berkepala 3 menandakan redirect, kode berkepala 4 menandakan error dengan request, sedangkan kode berkepala 5 mengindikasikan masalah dengan server.

Referensi:

- Author, "What is an API (Application Programming Interface)?", *Amazon*, [Online]. Available: <https://aws.amazon.com/what-is/api/>. [Accessed: 9 March 2025].
- The Postman Team, "What are the components of an API?", *Postman*, [Online]. Available: <https://blog.postman.com/what-are-the-components-of-an-api/>. [Accessed: 9 March 2025].
- David Demir, "What Are the Components of an API?", *Apidog*, [Online]. Available: <https://apidog.com/blog/what-are-the-components-of-an-api/>. [Accessed: 9 March 2025].
- Ronan McQuillan, "What are the Components of an API?", *Budibase*, [Online]. Available: <https://budibase.com/blog/inside-it/what-are-the-components-of-an-api/>. [Accessed: 9 March 2025].

2. Penjelasan diagram:

Diagram tersebut menggambarkan alur dan proses lengkap bagaimana seorang pengguna yang mengakses sebuah website dengan sistem database mendapatkan request datanya dari backend melalui frontend.

Frontend:

Client akan berinteraksi dengan tampilan frontend pada sebuah website yang biasanya tersusun atas 3 file utama (.html, .css, dan .js). HTML merupakan kerangka dasar dari sebuah website yang menentukan struktur dasar tampilan website, seperti adanya heading, button, gambar, dsb. CSS merupakan bagian yang menghias tampilan dari website agar menjadi lebih user-friendly dan menambahkan animasi atau appeal dari sebuah website seperti warna, ukuran, alignment, dan efek hover. JS atau Javascript merupakan bagian yang membuat tampilan website interaktif dengan

komponen internal maupun eksternal dari sebuah website. JS merupakan jembatan yang menghubungkan bagian frontend dengan bagian backend yang menyimpan semua informasi pada sebuah database.

Backend:

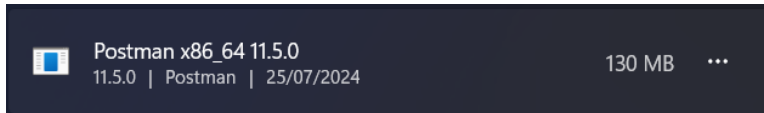
Bagian backend dari sebuah website menjadi tempat untuk mengimplementasikan sistem database kita yang menyimpan semua data. Umumnya backend terbagi menjadi 4 bagian (main, routes, controllers, dan repository). Bagian main akan mengatur navigasi request dari bagian satu ke bagian lainnya berdasarkan request dari frontend. Bagian ini berisi inisiasi framework dan dotenv, serta membangun koneksi dengan database. Bagian routes mengatur navigasi request berdasarkan endpoint untuk melakukan fungsi tertentu, sesuai dengan request yang diterima. Routes akan mengarahkan request ke controllers yang bertugas untuk memberikan instruksi berdasarkan fungsi masing-masing endpoint. Bagian controllers mengatur berhasil atau gagal nya sebuah request dengan mengembalikan status code dan message yang dapat diatur oleh pengelola database. Bagian repository bertanggung jawab untuk mengirimkan query ke database berdasarkan fungsi yang diatur pada bagian controllers. Bagian repository merupakan bagian yang terhubung langsung dengan database melalui command-command querynya.

Sistematika request:

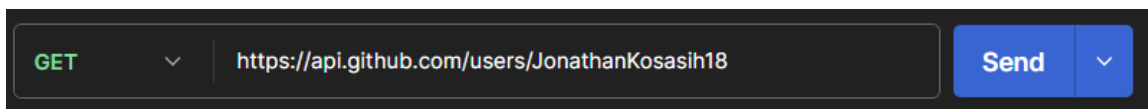
Saat mengakses website, client akan mengirimkan request ke frontend melalui protokol HTTP, kemudian frontend akan memberikan respons yang berisi file HTML, CSS, dan JS untuk menampilkan tampilan website pada perangkat client. Setelah perangkat memuat tampilan website, frontend akan membuat request kepada backend yang akan diterima pada bagian main. Bagian main akan memproses request tersebut berdasarkan endpoint yang diterima, lalu routes akan mengarahkan request ke controller berdasarkan endpoint tersebut. Controller akan mengatur instruksi untuk request dan bagian repository akan mengeksekusinya dalam bentuk query. Ketika eksekusi query pada database sudah selesai, hasilnya akan dilaporkan ke controller dan diolah dalam bentuk status code yang dikembalikan ke bagian main. Hasil request akan dikembalikan sebagai response kepada client dalam bentuk .json atau format backend umum lainnya.

PART 2: Praktik

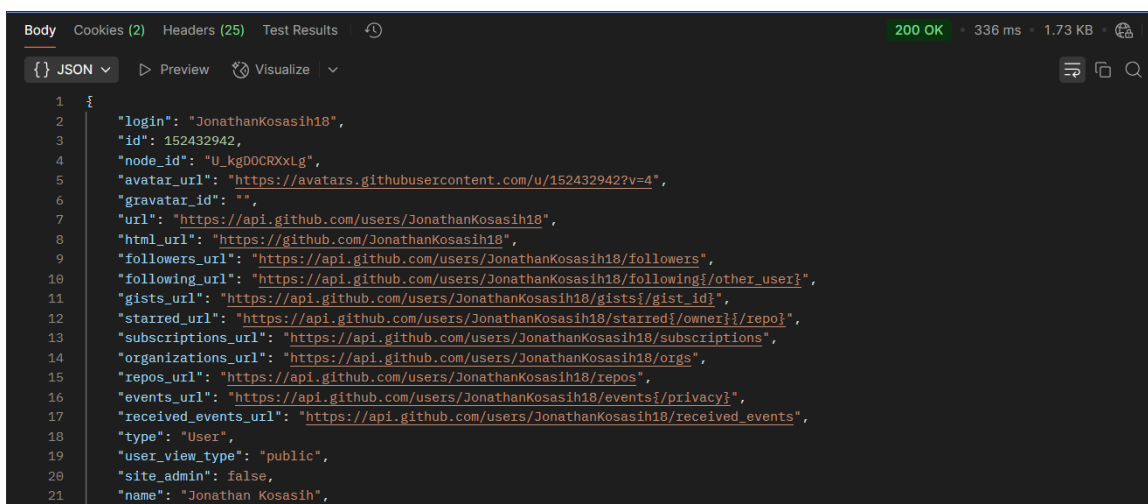
1. Postman sudah terinstall sebelumnya:



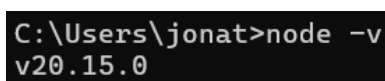
2. Screenshot pembuatan request API:



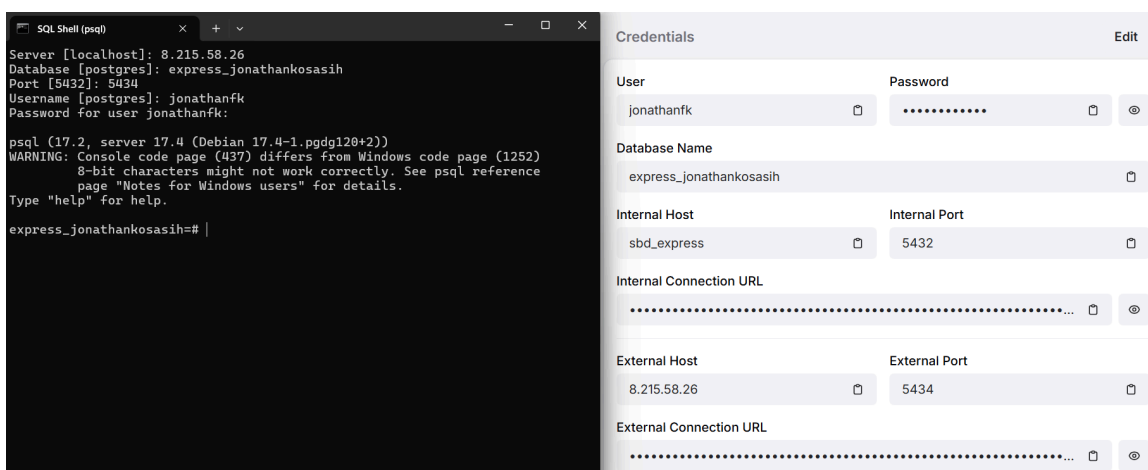
3. Screenshot response:



4. Node.js sudah terinstall sebelumnya:



5. Screenshot pembuatan database pada Alibaba Cloud:



6. Screenshot menjalankan query:

```
express_jonathankosasih=# CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION
express_jonathankosasih=# CREATE TABLE IF NOT EXISTS stores(
express_jonathankosasih(#   id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
express_jonathankosasih(#   name VARCHAR(255) NOT NULL,
express_jonathankosasih(#   address VARCHAR(255) NOT NULL,
express_jonathankosasih(#   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
express_jonathankosasih(# );
CREATE TABLE
express_jonathankosasih=# |
```

7. Pembuatan back end:

```
PS C:\Users\jonat\OneDrive\Documents\university\smt4\sbd\prak\modul5_sbd> npm i express pg dotenv
added 15 packages, and audited 85 packages in 3s

15 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\jonat\OneDrive\Documents\university\smt4\sbd\prak\modul5_sbd>

About to write to C:\Users\jonat\OneDrive\Documents\university\smt4\sbd\prak\modul5_sbd\package.json:
{
  "name": "modul5_sbd",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "description": ""
}

Is this OK? (yes)

npm notice
npm notice New major version of npm available! 10.7.0 -> 11.2.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.2.0
npm notice To update run: npm install -g npm@11.2.0
npm notice
PS C:\Users\jonat\OneDrive\Documents\university\smt4\sbd\prak\modul5_sbd>
```

server.js

```
const express = require('express');
require('dotenv').config();

const app = express();
const port = process.env.PORT || 3000;

app.use(express.json());

app.use("/store", require("./src/routes/store.route"));

app.listen(port, () => {
```

```
    console.log(`Server running on port ${port}`);  
  });
```

pg.database.js

```
require("dotenv").config();  
  
const { Pool } = require("pg");  
  
const pool = new Pool({  
  connectionString: process.env.PG_CONNECTION_STRING,  
  ssl: {  
    rejectUnauthorized: false  
  }  
});  
  
const connect = async () => {  
  try {  
    await pool.connect();  
    console.log("Database connected!");  
  } catch (error) {  
    console.error("Database connection error", error);  
  }  
}  
  
connect();  
  
const query = async (text, params) => {  
  try {  
    const result = await pool.query(text, params);  
    return result;  
  } catch (error) {  
    console.error("Database query error", error);  
  }  
}  
  
module.exports = { query };
```

store.repository.js

```
const db = require("../database/pg.database");  
  
exports.getAllStores = async () => {  
  try {
```

```
        const result = await db.query("SELECT * FROM stores");
        return result.rows;
    } catch (error) {
        console.error("Store repository error", error);
    }
};

exports.createStore = async (store) => {
    try {
        const result = await db.query(
            "INSERT INTO stores(name, address) VALUES($1, $2) RETURNING *",
            [store.name, store.address]
        );
        return result.rows[0];
    } catch (error) {
        console.error("Store repository error", error);
    }
};
```

store.controller.js

```
const storeRepository = require("../repository/store.repository");
const baseResponse = require("../utils/baseResponse.util");

exports.getAllStores = async (req, res) => {
    try {
        const stores = await storeRepository.getAllStores();
        baseResponse(res, true, 200, "Stores found", stores);
    } catch (error) {
        baseResponse(res, false, 500, "Error retrieving stores", error);
    }
};

exports.createStore = async (req, res) => {
    if (!req.body.name || !req.body.address) {
        return baseResponse(res, false, 400, "Missing store name or address", null);
    }
    try {
        const store = await storeRepository.createStore(req.body);
        baseResponse(res, true, 201, "Store created", store);
    }
};
```

```
catch (error) {
    baseResponse(res, false, 500, error.message || "Server error",
error);
}
};
```

store.route.js

```
const storeController = require("../controllers/store.controller");
const express = require("express");
const router = express.Router();

router.get("/getAll", storeController.getAllStores);
router.post("/create", storeController.createStore);
module.exports = router;
```

baseResponse.util.js

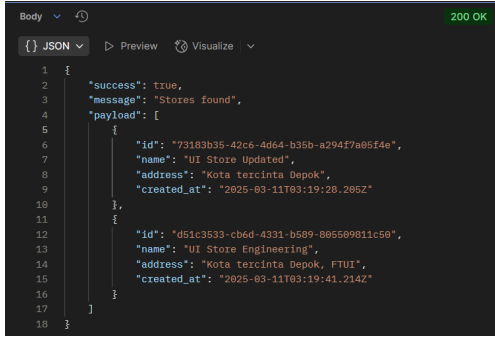
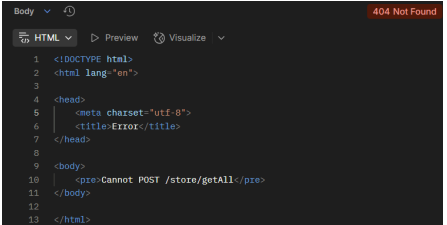
```
const baseResponse = (res, success, status, message, payload) => {
    return res.status(status).json({ success, message, payload });
}

module.exports = baseResponse;
```

.env

```
.env
1 PORT = 5434
2 PG_CONNECTION_STRING = postgres://jonathanfk:jambrongedan@8.215.58.26:5434/
  express_jonathankosasih?sslmode=disable
```

8. Screenshot response:

Method	Endpoint	Success	Failed
GET	/store/getAll		

POST	/store/create	Body JSON <pre> 1 { 2 "success": true, 3 "message": "Store created", 4 "payload": { 5 "id": "d51c3533-cbed-4331-b589-805509811c50", 6 "name": "UI Store Engineering", 7 "address": "Kota tercinta Depok, FTUI", 8 "created_at": "2025-03-11T03:19:41.214Z" 9 } 10 } </pre> 201 Created	Body JSON <pre> 1 { 2 "success": false, 3 "message": "Missing store name or address", 4 "payload": null 5 } </pre> 400 Bad Request
------	---------------	---	---